

C Programming Exercises

Name: PATHIPATI RUTHWIK

Roll No: CH.SC.U4CSE25230

Instructions: Read the code, understand the logic, and write the output in the space provided. Check your answer with the Expected Output.

Exercise 1: Hello World Program

Problem: Write a C program to print "Hello, World!" on the screen.

Code:

```
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Explanation:

1. `#include <stdio.h>` includes the standard input/output library.
2. `main()` is the entry point of every C program.
3. `printf()` is used to display text on the console.
4. `return 0;` indicates successful program termination.

Your Output:

Hello, World!

Expected Output:

Hello, World!

Exercise 2: Sum of Two Numbers

Problem: Write a C program to read two integers from the user and print their sum.

Code:

```
#include <stdio.h>

int main() {
    int a, b, sum;

    printf("Enter two numbers: ");

    scanf("%d %d", &a, &b);

    sum = a + b;

    printf("Sum = %d", sum);

    return 0;
}
```

Explanation:

1. `int a, b, sum;` declares three integer variables.
2. `scanf()` reads user input. `&a` is the address of variable `a`.
3. `sum = a + b;` performs addition.
4. `printf()` displays the result using `%d` format specifier for integers.

Your Output:

```
Enter two numbers: 12 28
Sum = 40
```

Expected Output:

Enter two numbers: 5 7

Sum = 12

Exercise 3: Check Even or Odd

Problem: Write a C program to check whether a given number is even or odd using if-else.

Code:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 2 == 0) {
        printf("%d is Even", n);
    } else {
        printf("%d is Odd", n);
    }
    return 0;
}
```

Explanation:

1. The modulo operator % gives the remainder.
2. If $n \% 2 == 0$, the number is perfectly divisible by 2, so it's Even.
3. Otherwise, the if-else branch prints Odd.
4. This introduces conditional control flow.

Your Output:

Enter a number: 28
28 is Even

Expected Output:

Enter a number: 13
13 is Odd

Exercise 4: Factorial using For Loop

Problem: Write a C program to find the factorial of a number using a for loop.

Code:

```
#include <stdio.h>

int main() {
    int n, i;
    long fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("Factorial of %d = %ld", n, fact);
    return 0;
}
```

```
}
```

Explanation:

1. Factorial $n! = 1 \times 2 \times 3 \times \dots \times n$
2. The for loop runs from $i = 1$ to n .
3. In each iteration, fact is multiplied by i .
4. long is used to store larger factorial values.
5. Example: $5! = 120$

Your Output:

Enter n: 5
Factorial of 5 = 120

Expected Output:

Enter n: 5
Factorial of 5 = 120

Exercise 5: Sum of First N Natural Numbers

Problem: Write a C program to calculate the sum of first N natural numbers using a while loop.

Code:

```
#include <stdio.h>

int main() {
    int n, i = 1, sum = 0;

    printf("Enter N: ");
    scanf("%d", &n);

    while(i <= n) {
        sum += i;
        i++;
    }

    printf("Sum = %d", sum);
    return 0;
}
```

Explanation:

1. We initialize $\text{sum} = 0$ and $i = 1$.
2. The while loop continues as long as $i \leq n$.
3. $\text{sum} += i$ is shorthand for $\text{sum} = \text{sum} + i$.
4. $i++$ increments the counter each iteration.
5. For $N=10$, the sum is 55.

Your Output:

Enter N: 10

Sum = 55

Expected Output:

Enter N: 10

Sum = 55

Exercise 6: Multiplication Table - Nested Loop

Problem: Write a C program to print the multiplication table of a given number.

Code:

```
#include <stdio.h>

int main() {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);

    for(i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

Explanation:

1. The for loop iterates 10 times, from $i=1$ to $i=10$.
2. In each iteration, $n * i$ is calculated and printed.
3. `\n` creates a new line for each result.
4. This demonstrates a simple iterative loop in practice.

Your Output:

Enter a number: 3

$3 \times 1 = 3$

$3 \times 2 = 6$

$3 \times 3 = 9$

$3 \times 4 = 12$

$3 \times 5 = 15$

$3 \times 6 = 18$

$3 \times 7 = 21$

$3 \times 8 = 24$

$3 \times 9 = 27$

$3 \times 10 = 30$

Expected Output:

Enter a number: 7

$7 \times 1 = 7$

$7 \times 2 = 14$

$7 \times 3 = 21$

$7 \times 4 = 28$

$7 \times 5 = 35$

$7 \times 6 = 42$

$7 \times 7 = 49$

$7 \times 8 = 56$

$7 \times 9 = 63$

$$7 \times 10 = 70$$

Exercise 7: Reverse a Number - While Loop

Problem: Write a C program to reverse the digits of a given integer.

Code:

```
#include <stdio.h>

int main() {
    int n, rev = 0, digit;

    printf("Enter a number: ");
    scanf("%d", &n);

    while(n > 0) {
        digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }

    printf("Reversed = %d", rev);
    return 0;
}
```

Explanation:

1. $n \% 10$ extracts the last digit.
2. $rev = rev * 10 + digit$ builds the reversed number.
3. $n = n / 10$ removes the last digit.

4. The loop continues until n becomes 0.

5. Example: 1234 -> 4321

Your Output:

Enter a number: 1228

Reversed = 8221

Expected Output:

Enter a number: 1234

Reversed = 4321

Exercise 8: Check Prime Number

Problem: Write a C program to check if a number is Prime using a for loop.

Code:

```
#include <stdio.h>

int main() {
    int n, i, isPrime = 1;
    printf("Enter n: ");
    scanf("%d", &n);

    if(n <= 1) {
        isPrime = 0;
    } else {
```

```
for(i = 2; i * i <= n; i++) {  
    if(n % i == 0) {  
        isPrime = 0;  
        break;  
    }  
}  
  
if(isPrime)  
    printf("%d is Prime", n);  
else  
    printf("%d is Not Prime", n);  
  
return 0;  
}
```

Explanation:

1. A prime number is only divisible by 1 and itself.
2. We check divisors from 2 up to $\sqrt{n}$ ($i*i \leq n$).
3. If any divisor is found, isPrime is set to 0 and we break.
4. The flag isPrime determines the final output.

Your Output:

Enter n: 3
3 is Prime

Expected Output:

Enter n: 29

29 is Prime

Exercise 9: Fibonacci Series using Recursion

Problem: Write a C program to print the Fibonacci series up to N terms using recursion.

Code:

```
#include <stdio.h>

int fib(int n) {
    if(n <= 1)
        return n;
    return fib(n-1) + fib(n-2);
}

int main() {
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");
    for(i = 0; i < n; i++) {
        printf("%d ", fib(i));
    }
    return 0;
}
```

Explanation:

1. Fibonacci: 0, 1, 1, 2, 3, 5, 8...
2. fib() is a recursive function that calls itself.
3. Base case: if $n \leq 1$, return n.
4. Recursive case: $\text{fib}(n-1) + \text{fib}(n-2)$.
5. main() calls fib() for each term from 0 to n-1.

Your Output:

Enter number of terms: 3
Fibonacci Series: 0 1 1

Expected Output:

Enter number of terms: 8
Fibonacci Series: 0 1 1 2 3 5 8 13

Exercise 10: GCD using Recursion - Euclidean Algorithm

Problem: Write a C program to find the GCD (Greatest Common Divisor) of two numbers using recursion.

Code:

```
#include <stdio.h>

int gcd(int a, int b) {
    if(b == 0)
        return a;
    return gcd(b, a % b);
}

int main() {
    int x, y;

    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);

    printf("GCD of %d and %d = %d", x, y, gcd(x, y));

    return 0;
}
```

Explanation:

1. This uses the Euclidean Algorithm recursively.
2. $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$
3. Base case: if $b == 0$, return a .
4. The function keeps calling itself with smaller values until b becomes 0.
5. Example: $\text{gcd}(48, 18) = 6$

Your Output:

Enter two numbers: 12 28

GCD of 12 and 28 = 4

Expected Output:

Enter two numbers: 48 18

GCD of 48 and 18 = 6